

# Predictive Storage Caching and Tiered Data Migration

**GROUP 10**

---

Nilay Sadanand Borkar: 2023A7PS0401G

Ishan Kaustubh Mehta: 2023A7PS0379G

# Outline

- Our Goals
- Background Needed
- Work Done

---

# Overview

There is an upper bound on the amount of the Cache memory available to us, and this memory is not enough to hold all the KV Cache generated during the prefill phase.

Thus, we use tiered caching. But there is catch! Moving KV pairs between different tiers of storage comes with a **time cost**. Also since cache fetching operations are **reactive**, meaning the cache is fetched from storage after the Model requests it, we halt LLM's forward pass while data is fetched from disk to CPU.

- We attempted to run some experiments how LMCache can help in migration of cache between tiered storages
- Then we tried to build an RL model that reactively learns to migrate the predicted chunks of cache in the *warmer* tiers.

# Background & Research

# Related Works

**HVAC (Distributed Aggregate Caching):** \* A POSIX-compliant client-server model that intercepts read requests without altering the DL pipeline.

- Checks local aggregate caches via RPC before loading data from the Parallel File System (PFS).
- Stores the KV pairs in cache for future fetching.

**FedCaSe (Resource Optimization):**

- Replaces monetary cost models with a focus on system resource and training time reduction.
- Utilizes a **Reverse Optimization (RO) model** to balance I/O utility (speed) and Model utility (accuracy), prioritizing fast-memory clients.



# Related Works

## SHADE (Semantic Value Caching):

- Addresses the caching gap by identifying that not all training data is equally valuable.
- **Priority-based Adaptive Sampling (PADS)** to cache "hard-to-learn" (high-loss) samples.
- Maintains a **Ghost Cache** to store relative importance scores even after memory eviction.
- *Impact:* Artificially creates temporal locality, cache hits up to 4.5x and throughput by 3.3x.

## LMCache (Tiered Storage for LLMs):

- Shifts the focus from training data I/O to inference computation bottlenecks.
- Stores and migrates **Key-Value (KV) Caches** across tiered storage architectures (GPU Memory -> CPU RAM -> NVMe Disk).
- Predictively retrieves intermediate attention states for overlapping prompts to completely bypass redundant GPU recalculations.

Work Done

# What we learnt

So, the logical step is to know what part of the cache will be needed, so that you can preload the KV pairs from the cache . Now the problem with this is that we don't know what the next prompt will be.

User prompts also don't necessarily follow the principles of temporal or spatial locality since the next query or question can be from any part of the existing conversation or a totally new question.

How do we deal with this?



# Work Done in Milestone 3

# Earlier Plan: Problem!

- LSTM is “**predictive**” - requires **complex periodic training** to remain accurate, and **computationally expensive** requiring high memory bandwidth too.
- Access patterns can shift suddenly, but the model could remain **stale**.
- LSTMs can **fail to capture long-term dependencies** over thousands of accesses - common problem in large scale workloads.

# New Plan: Reinforcement Learning!

- RL is “**adaptive**” - a real-time decision making framework, learns through trial & error giving real-time feedback.
- It can be designed with our needs - **reward function** - penalising wasted migration & rewarding TTFT improvements.

# RL-Driven Adaptive KV-Cache Tier Migration

**The Engine:** We leverage a lightweight **Multi-Layer Perceptron (MLP)** trained via **PPO (Proximal Policy Optimisation)**, executing in microseconds so as not to add decision-making latency.

We developed `HardwareCacheEnv`—a custom Gymnasium environment that controls actual data movement on physical hardware (RTX 3050):

- Tier L1 (GPU VRAM): The fastest layer. Where the LLM accesses tensors instantly.
- Tier L2 (Pinned CPU RAM): Intermediate layer. Fast PCIe transfer to GPU.
- Tier L3 (NVMe Disk): The slowest layer, offering massive capacity. The RL Agent strictly manages the proactive migration of chunks from L3 to L2 to prepare them for L1 access.

# The Agent (Observation Space)

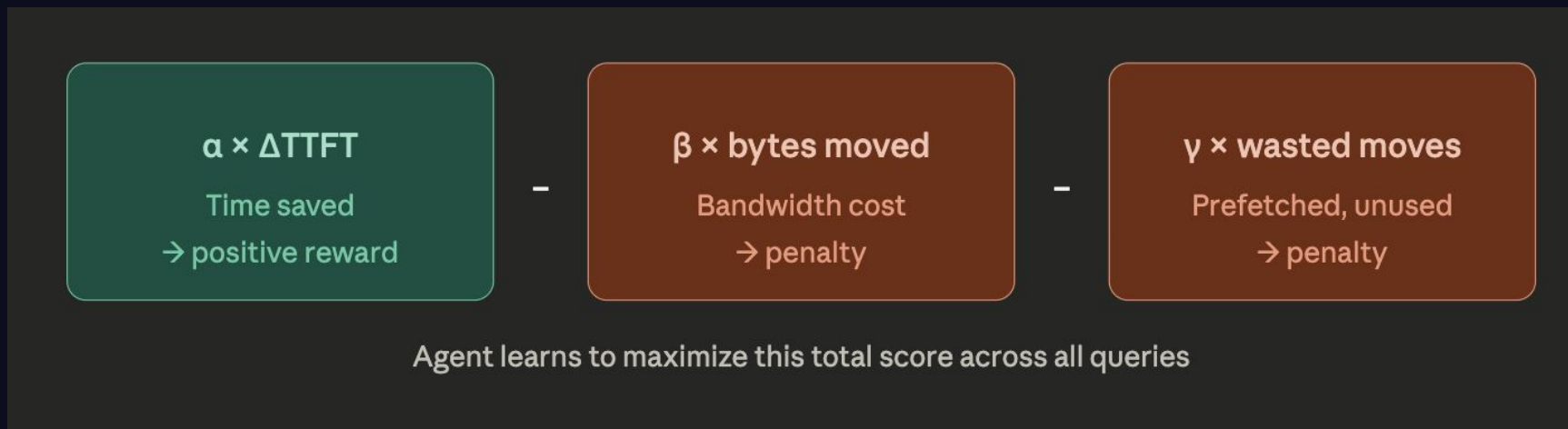
How well RL knows what to fetch? It looks at the **403-dim vector**.

- **Semantic Awareness (384-d)**: We encode the incoming query using a CPU-based `SentenceTransformer`(MiniLM-L6). The agent "reads" the query to understand the context.
- **System Awareness (3-d)**: Real-time normalized utilization metrics of the GPU, CPU, and Disk tiers.
- **Temporal Awareness (16-d)**: Recency scores tracking the top 16 candidate chunks sitting in the L3 Disk cache.

## Action Space (Decision to Migrate)

- Agent outputs 16-dim binary vector (`Multibinary(16)`) -> for each of top 16 candidates in L3.
- Selected chunks synchronously trigger disk-to-CPU read operation -> now in L2!

# Reward Function: Penalty for Being Wrong



- Positively reward if we prefetched correctly, reducing latency
- ‘Structural’ penalty to the number of MBs moved to warmer tier.
- Harsh penalty to the wastage of bandwidth, because agent moved useless data

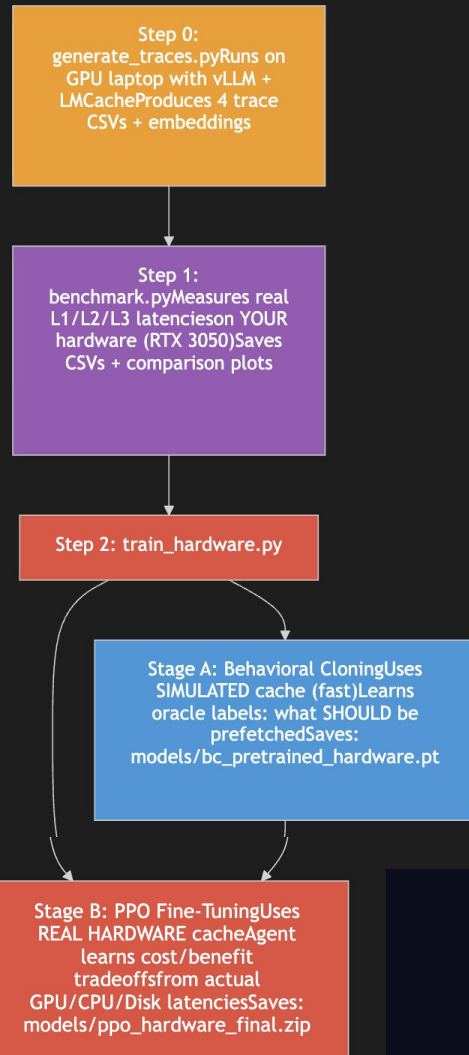
# Two-Stage Training Pipeline

## 1. Behavioral Cloning (BCE Loss)

- Agent mimics an “**Oracle**”, i.e. it knows the future queries & prefetches accordingly
- This teaches the RL to generate the `Multibinary` vectors locally using BCE loss, proves fast & steady.

## 2. PPO Fine-tuning on Hardware

- Pre-trained Brain now runs on physical `HardwareCacheEnv`
- Trains on true latencies, disk cache migrations, optimising behaviour against real-time.





# Running the Experiment

# Generating traces

Traces are the logs of which exact KV pairs each query requires. They map these KV pairs to chunks and keep track of them.

## Why We Need Traces

- **Enables Offline RL Training:** Trains agents in simulated environments to save on expensive, live GPU resources.
- **Realistic Constraints:** Captures physical hardware limits and performance bounds to mirror real-world production.
- **Accelerates Iteration:** Replays traces to learn optimal prefetching policies thousands of times faster than real-time execution.

# Generating traces

How are traces generated?

- **Real-World Execution:** Processes sample prompts through a vLLM engine with LMCache to trigger actual GPU, CPU, and Disk memory movements.
- **Latency & Memory Logging:** Measures the exact Time-To-First-Token (TTFT) and memory footprint to establish "ground truth" hardware performance.
- **Data Chunking:** Algorithmically divides token sequences into fixed-size cache chunks, separating shared data (RAG docs) from unique user prompts.
- **CSV Serialization:** Saves the timing and chunk metadata into a deterministic trace file, allowing simulators to replay the session without active LLM inference.

```
query_id,query_text,input_tokens,chunk_ids_needed,shared_chunk_ids,unique_chunk_ids,num_chunks
1,Who is the founder of Microsoft?,7,[0],[],[0],1
2,What is the chemical symbol for sodium?,8,[1],[],[1],1
3,How many players are there in a baseball team?,10,[2],[],[2],1
4,What year did World War II end?,8,[3],[],[3],1
5,What is the tallest species of tree?,8,[4],[],[4],1
6,Who wrote The Divine Comedy?,6,[5],[],[5],1
7,What is the hardest rock type?,7,[6],[],[6],1
8,How many hearts does a squid have?,8,[7],[],[7],1
9,What is the main ingredient in guacamole?,10,[8],[],[8],1
10,Who developed the polio vaccine?,7,[9],[],[9],1
11,What is the longest-running Broadway show?,8,[10],[],[10],1
12,What is the square of 25?,9,[11],[],[11],1
13,Who was the first woman to win a Nobel Prize?,11,[12],[],[12],1
14,What is the currency of Brazil?,7,[13],[],[13],1
15,What gas do plants absorb during photosynthesis?,9,[14],[],[14],1
16,Who directed the movie Jaws?,7,[15],[],[15],1
17,What is the largest internal organ in the human body?,11,[16],[],[16],1
18,How many elements are in the periodic table?,9,[17],[],[17],1
19,What is the freezing point of mercury in Celsius?,10,[18],[],[18],1
20,Who painted The School of Athens?,7,[19],[],[19],1
21,What is the smallest unit of life?,8,[20],[],[20],1
22,How many time zones are there in Russia?,9,[21],[],[21],1
23,What is the main language spoken in Argentina?,9,[22],[],[22],1
24,Who invented the diesel engine?,6,[23],[],[23],1
25,What is the diameter of Earth in kilometers?,9,[24],[],[24],1
```

# Simulation Pipeline

- **Purpose:** Enables RL training without physical hardware by mimicking system latency and memory constraints.
- **Core Architecture:** A dual-component Python system using a specialized Cache Simulator and a Gymnasium Wrapper.
- **Input Data:** Utilizes pre-generated Trace CSVs (50 sequential user queries per episode).
- **Key Outcome:** Evaluates "Time-To-First-Token" (TTFT) based on simulated millisecond bottlenecks.

# Hardware Pipeline

- **The Concept:** A "drop-in" replacement for mathematical simulations that swaps theoretical latency for actual physical execution on hardware.
- **The 3-Tier Physical Memory:**
  - **L1 (GPU VRAM):** Real `torch.cuda.FloatTensor` objects for instant access.
  - **L2 (CPU Pinned RAM):** Tensors locked in physical memory using `pin_memory()` to enable high-speed Direct Memory Access (DMA).
  - **L3 (NVMe Disk):** Data stored as raw `.bin` files to mimic production C++ engine serialization.
- **The Goal:** Forces the AI to navigate real-world bottlenecks like PCIe bus saturation and SSD I/O limits that math alone cannot predict.

# Testing

**Standardized Proving Ground:** Validates the trained agent against identical user traces under locked hardware constraints to quantify Time-To-First-Token (TTFT) savings.

**The Competition (Baselines):**

- **No-Cache:** Baseline "worst-case" (recompute everything).
- **LRU (Passive):** Industry standard (passive eviction, no future-prediction).
- **Oracle (Ideal):** An "impossible" peak-performance limit used to measure how close the AI gets to "perfection."

**The tests are done over 5 tasks, Prefix, RAG, No context, Multiturn and interleaved.**

# Results



# Why Multi-turn chat had weird results?

LMCache operates on **\*\*fixed-size token chunks\*\*** (CHUNK\_SIZE = 256 tokens). For turn N:

- **\*\*Reusable prefix\*\***: Chunks from the conversation history that were cached in turn N-1
- **\*\*New suffix\*\***: The new 'User: Q\_N\nAssistant: Fake\_A\_N\nUser: elaborate on: Q\_N' appended at the end
- **\*\*Partial chunk boundary issue\*\***: If the new content starts mid-chunk, the entire chunk must be recomputed

The latency trace tells the real story:

| Turn Range     | Avg Latency       | Input Tokens | GPU KV Usage |
|----------------|-------------------|--------------|--------------|
| Q1-Q7          | ~0.16s            | 65-226       | 1.6%-5.5%    |
| Q8-Q31         | ~0.17s            | 256-884      | 6.2%-21.6%   |
| <b>Q32-Q35</b> | <b>0.66-0.99s</b> | 915-999      | 22.3%-24.4%  |
| Q36-Q50        | ~0.56s            | 1032-1418    | 25.2%-34.6%  |

At **Q32 the latency jumps 4x** (from ~0.19s to ~0.67–0.99s), but the cache shows the queries are still marked "Warm." This is because:

1. **Prompt is ~915 tokens → ~3.5 chunks cached**, but the new tail content always needs fresh computation
2. **The prefill cost scales with total prompt length**, not just the new portion, because vLLM still needs to process attention over all past tokens even if KV cache is reloaded
3. The model has crossed a performance cliff where the quadratic attention cost over ~900+ tokens starts dominating

### 3a. Unrealistically short history

Real multi-turn chats would have assistant responses of 50–200+ tokens each. The fake 8-word responses mean the history grows **much slower** than a real conversation. By turn 50, you only have ~1418 tokens instead of potentially 5000–10000+ tokens in real.

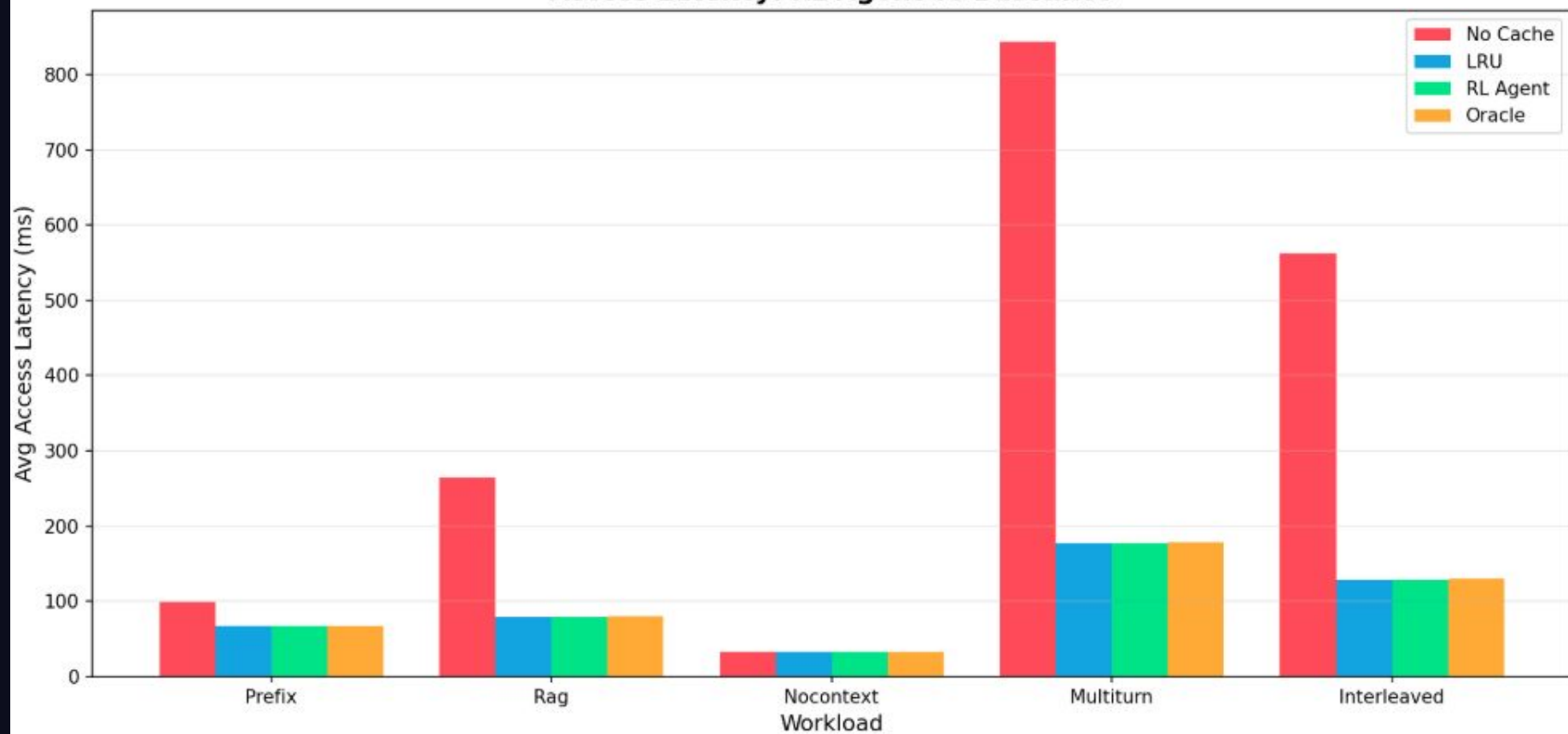
**Impact**: The experiment is testing a **much weaker** version of the multi-turn problem than reality. Yet it's performing poorly.

### 3b. No KV continuity with prior generations

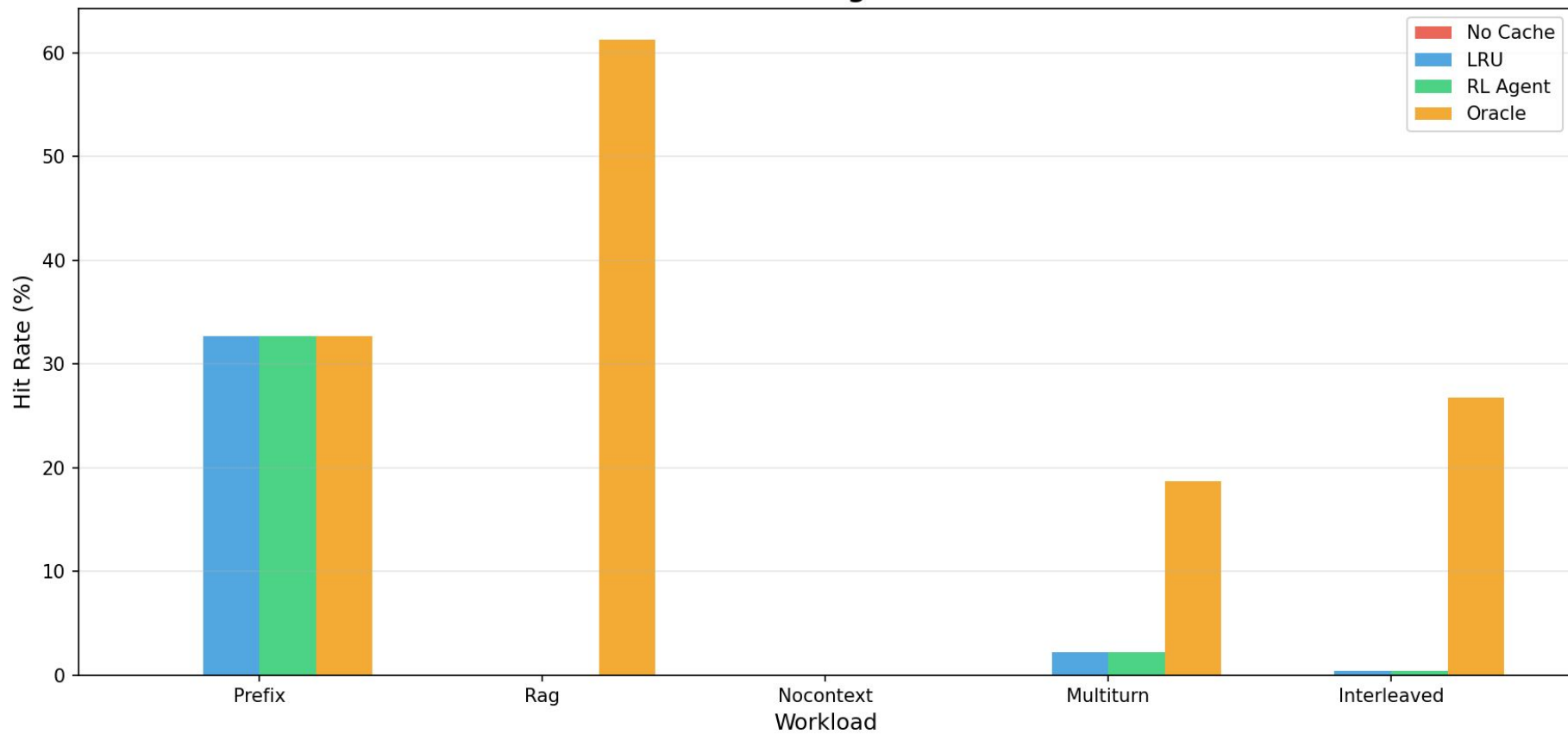
In a real multi-turn system, the KV cache from generating turn N's response would be **kept in-place** for turn N+1. The engine would just append the new user message and continue decoding. But in this experiment:

1. Each prompt is sent as an entirely **new, independent request**
  2. The engine discards internal KV state after each `llm.generate()` call
  3. LMCache can only help by **reloading** cached prefix chunks from disk/CPU – not by keeping live GPU KV state
- This means you're measuring **cache restore latency + re-attention cost**, not true KV cache reuse.

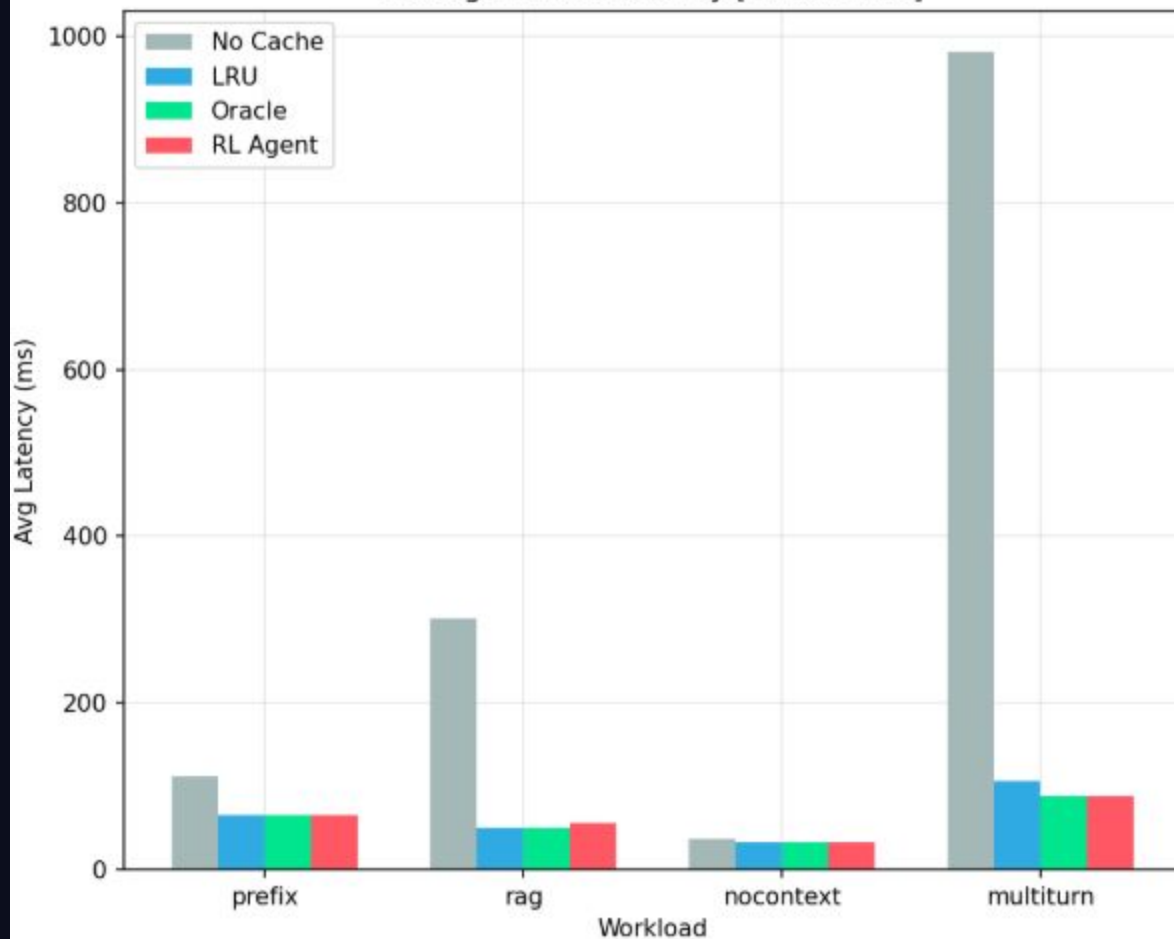
**Access Latency: RL Agent vs Baselines**



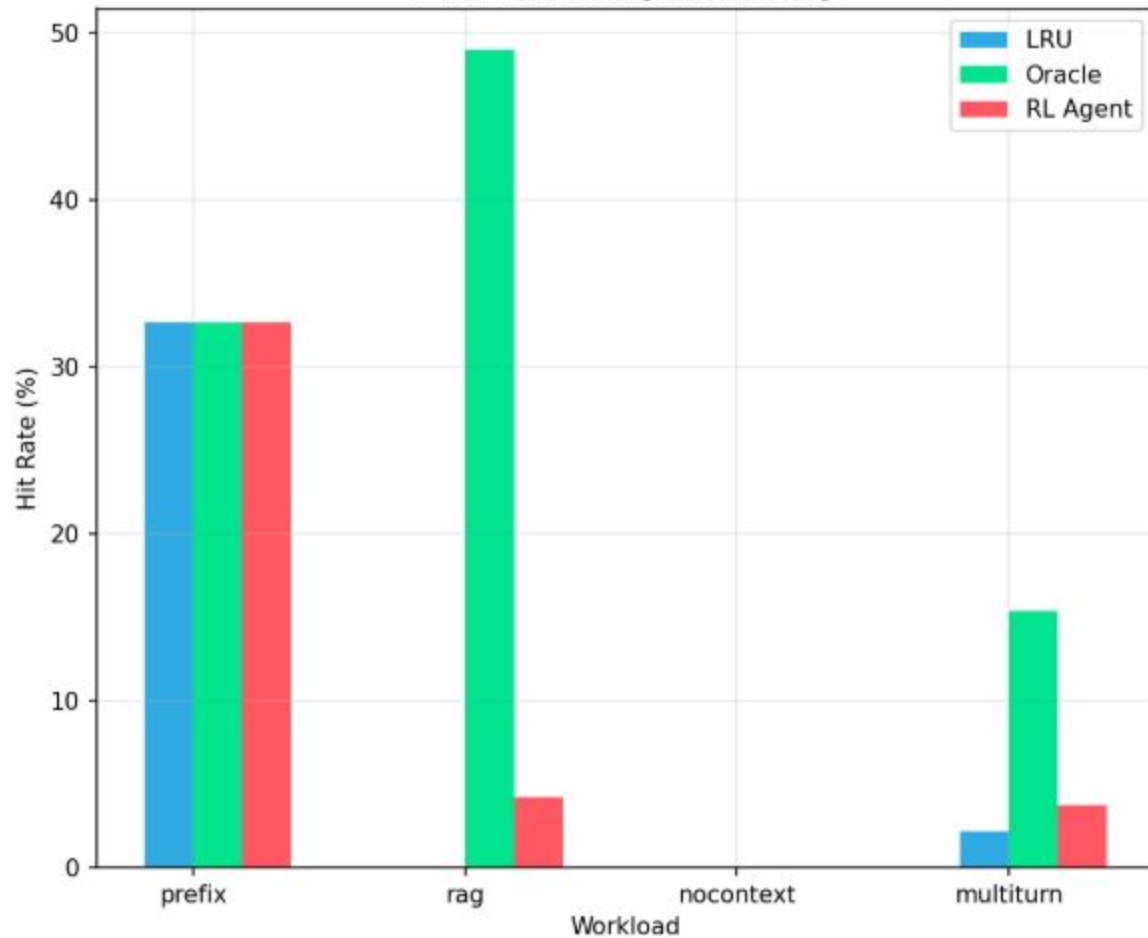
**Cache Hit Rate: RL Agent vs Baselines**



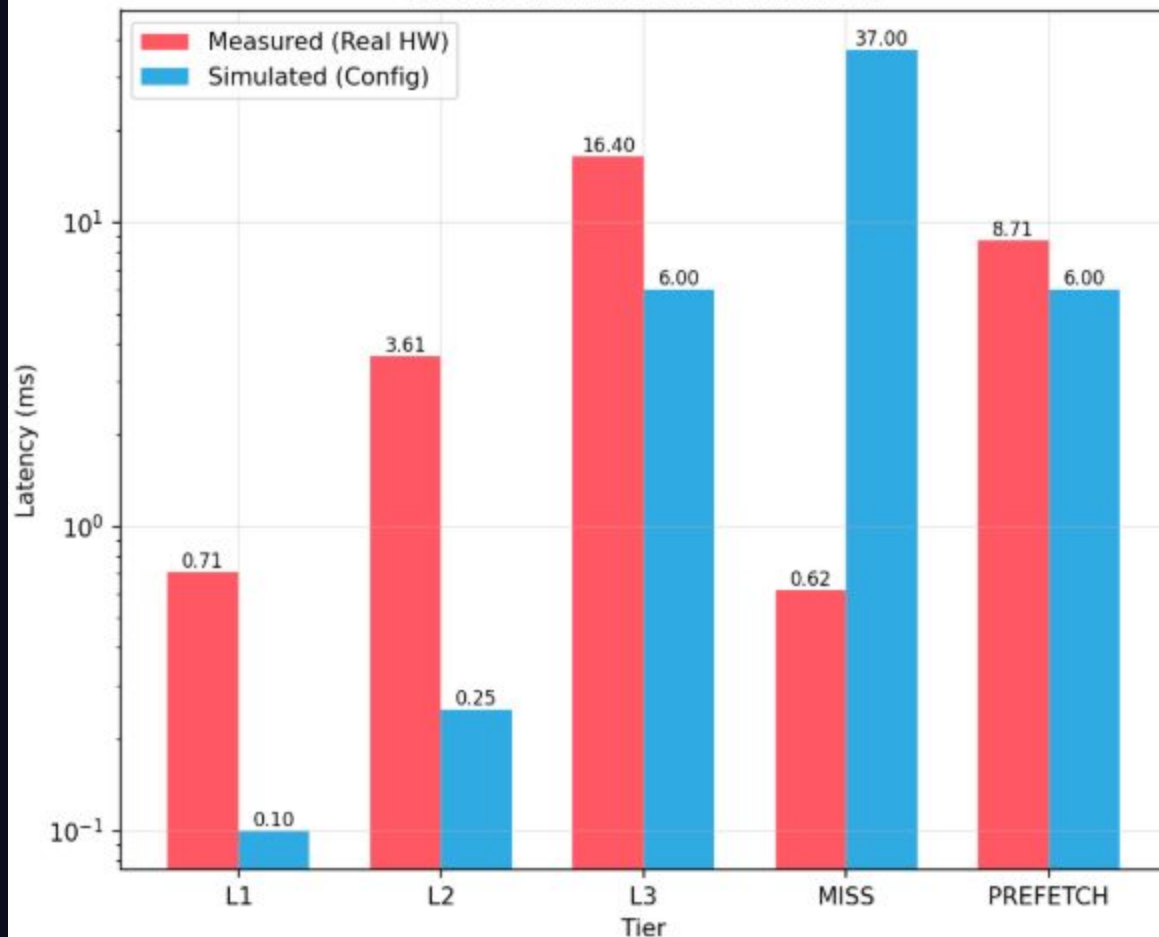
Average Access Latency [HARDWARE]



Cache Hit Rate [HARDWARE]



Measured vs Simulated Latencies



# Further Experimentation

Further, after running these experiments, we decided that we should run the model on actual traces, instead of the simulated traces.

However LMcache doesn't make the chunks it uses public, so we had to allocate tiers and migrate the chunks using vLLM without LMCache.



# Stage 3

Real traces

Binary Cross Entropy loss (Stage A)

PPO fine tuning (Stage B)

Action Space: Only Prefetching.

L1: 3Mb

L2: 4.5Mb

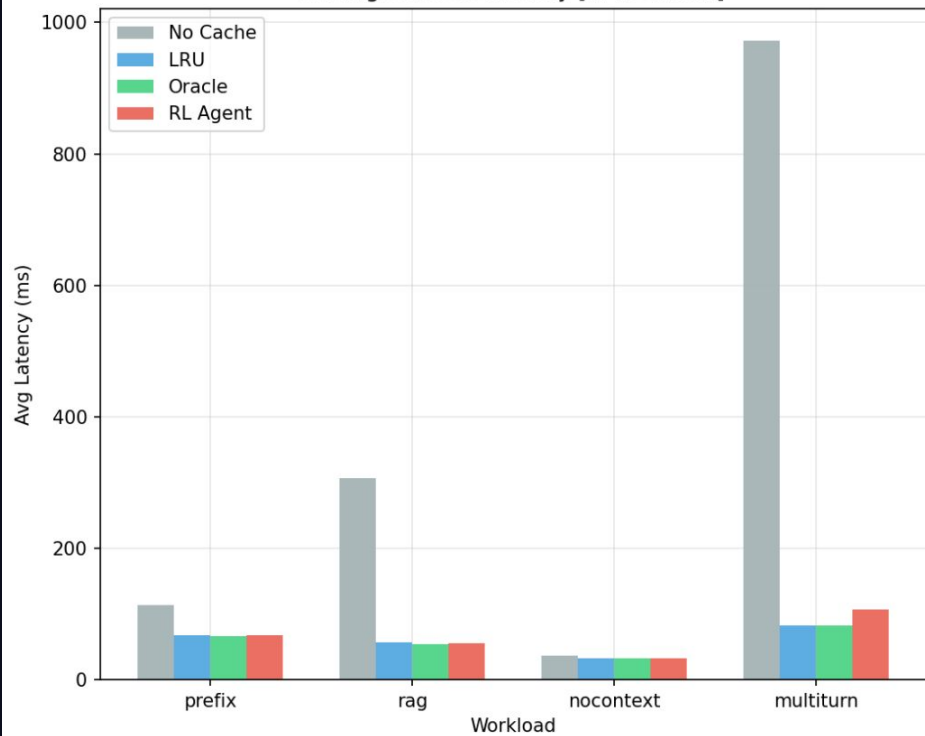
Chunk Size: 3MB



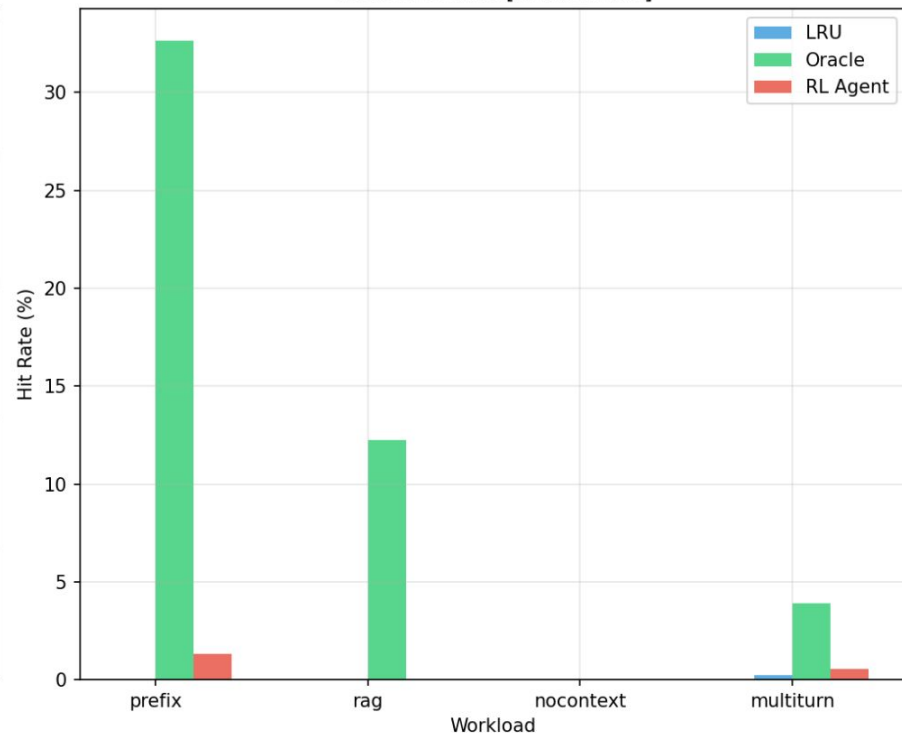
## HARDWARE EVALUATION SUMMARY

| Workload  | Strategy | Avg Lat (ms) | Hit Rate | Speedup |
|-----------|----------|--------------|----------|---------|
| prefix    | No Cache | 113.41       | 0.0      | 0.59x   |
| prefix    | LRU      | 67.04        | 0.0      | 1.00x   |
| prefix    | Oracle   | 65.76        | 32.7     | 1.02x   |
| prefix    | RL Agent | 67.94        | 1.3      | 0.99x   |
| rag       | No Cache | 307.17       | 0.0      | 0.19x   |
| rag       | LRU      | 57.40        | 0.0      | 1.00x   |
| rag       | Oracle   | 54.54        | 12.2     | 1.05x   |
| rag       | RL Agent | 55.52        | 0.0      | 1.03x   |
| nocontext | No Cache | 37.06        | 0.0      | 0.88x   |
| nocontext | LRU      | 32.65        | 0.0      | 1.00x   |
| nocontext | Oracle   | 32.66        | 0.0      | 1.00x   |
| nocontext | RL Agent | 32.86        | 0.0      | 0.99x   |
| multiturn | No Cache | 972.90       | 0.0      | 0.08x   |
| multiturn | LRU      | 82.45        | 0.2      | 1.00x   |
| multiturn | Oracle   | 82.31        | 3.9      | 1.00x   |
| multiturn | RL Agent | 106.16       | 0.6      | 0.78x   |

Average Access Latency [HARDWARE]



Cache Hit Rate [HARDWARE]



# Stage 4

Real traces

Binary Cross Entropy loss (Stage A)

PPO fine tuning (Stage B)

Action Space: Both Eviction and Prefetching.

L1: 30Mb

L2: 60Mb

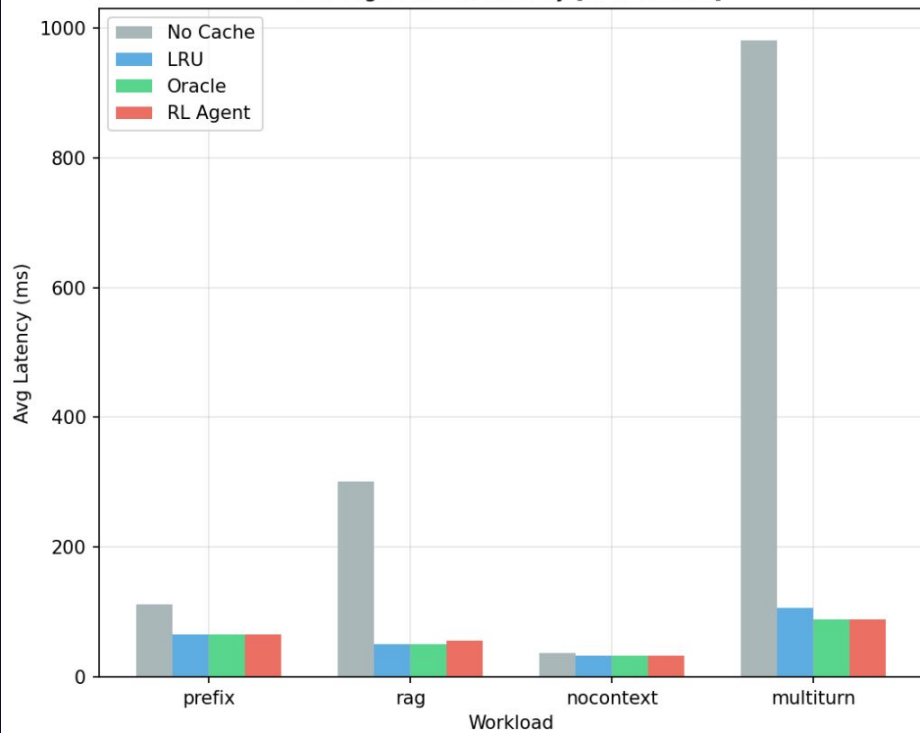
Chunk size: 3mb



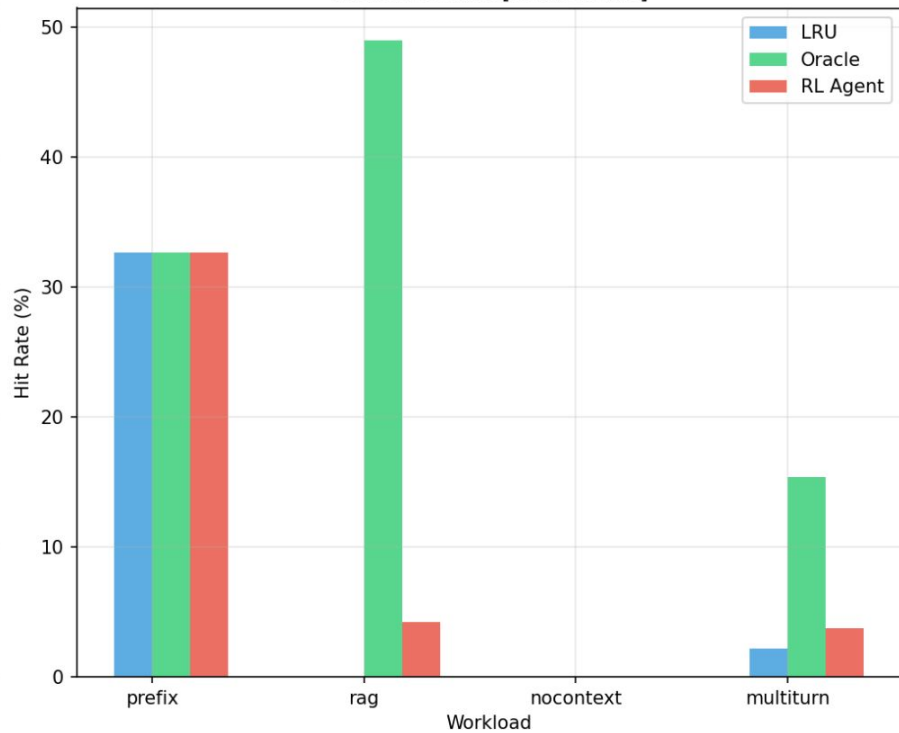
## HARDWARE EVALUATION SUMMARY

| Workload  | Strategy | Avg Lat (ms) | Hit Rate | Speedup |
|-----------|----------|--------------|----------|---------|
| prefix    | No Cache | 111.61       | 0.0      | 0.60x   |
| prefix    | LRU      | 66.44        | 32.7     | 1.00x   |
| prefix    | Oracle   | 64.89        | 32.7     | 1.02x   |
| prefix    | RL Agent | 65.11        | 32.7     | 1.02x   |
| rag       | No Cache | 296.50       | 0.0      | 0.12x   |
| rag       | LRU      | 36.74        | 85.8     | 1.00x   |
| rag       | Oracle   | 36.68        | 85.8     | 1.00x   |
| rag       | RL Agent | 36.76        | 85.8     | 1.00x   |
| nocontext | No Cache | 37.06        | 0.0      | 0.86x   |
| nocontext | LRU      | 31.88        | 0.0      | 1.00x   |
| nocontext | Oracle   | 31.92        | 0.0      | 1.00x   |
| nocontext | RL Agent | 31.90        | 0.0      | 1.00x   |
| multiturn | No Cache | 963.28       | 0.0      | 0.09x   |
| multiturn | LRU      | 86.24        | 36.5     | 1.00x   |
| multiturn | Oracle   | 61.71        | 66.3     | 1.40x   |
| multiturn | RL Agent | 63.68        | 59.6     | 1.35x   |

Average Access Latency [HARDWARE]



Cache Hit Rate [HARDWARE]



# Similar Ideas

## **STRCacheML: A Machine Learning-Assisted Content Caching Policy** (2024)

- It estimates the likelihood of cache objects being requested again based on available platform attributes, aimed at streaming services (using IMDb data)

## **ML-CLOCK: Efficient Page Cache Algorithm** (2021)

- It uses a machine learning model (Perceptron) to learn I/O patterns from workloads, determining which pages to evict based on future probability rather than just recent usage

## **LeCaR: Learning-to-Cache with Reinforcement Learning** (2018)

- It uses a Bayesian learning-based algorithm (a form of ML) to dynamically adjust between different caching strategies (like LRU and LFU) based on the current workload.



**Questions?**





**Thankyou**